

- count is a special task where we just consider a number as “a set of digits” so count in a float, fraction and scientific notation is as the same as in an integer.
- –: not applicable.
 - In fraction and scientific notation, the digit concept is not well-defined so the tasks about digit (digit-compare, digit-add, get-digit and length) are not applicable.
 - Floordiv and mod is only defined on integer.
 - Integer and float do not need to be further converted to float. Similarly, scientific has no need to converted to scientific.

A.1.6 EASY/HARD SPLIT OF NUPA TASKS

We divide the tasks into easy and hard as shown in Table 6, where the hard tasks marked as H with maximal test digit as 20 and the easy tasks marked as E with maximal test digit as 100.

Table 6: Tasks can be divided into Easy and Hard.

	Elementary arithmetic						Comparison		Digit Understanding							Conversion		
	Add	Sub	Multiply	Truediv	Floordiv	Mod	Max	Min	Digit Max	Digit Min	Digit Add	Get Digit	Length	Count		To Float	To Scientific	Sig. Fig.
Integer	H	H	H	H	H	H	E	E	E	E	E	E	E	E		E	E	E
Float	H	H	H				E	E	E	E	E	E	E				E	E
Fraction	H	H	H	H			H	H								H		
Scientific	H	H	H				E	E								E		

A.1.7 PREPROCESS AND QUESTION GENERATION FOR NUPA TASKS

We define the length of a number as the number of digits in the *longest part* of a number. The “integer” part and “decimal” part of a float (as well as the significand of a scientific notation), the “numerator” and “denominator” of a fraction, the “exponent” of a scientific notation are considered as different “parts”. In order to generate a pair of numbers with the larger length L , we first generate a L -length number and then generate a l -length number where l follows a uniform distribution from $L/2$ to L . If the operation is commutative, we swap the two numbers with probability 0.5.

After we select two random numbers, we have some preprocessing to generate the final questions:

- For “Multiply”, the difficulty also affected by the shorter number severely, so we split the task into two sub-tasks as “multiply-hard” and “multiply-easy”. For the hard subset, we require that the shorter number must be longer than half of the longer one. For an easy subset, we require that the length of the shorter number is less than 3, so that the complexity is $O(n)$ instead of $O(n^2)$. And because the addition of fractions also involves multiplication, we also add an add-easy for this task in the same way.
- For “max” and “min” tasks, we additionally provide a harder version. For Integers and floats, we make sure that two compared numbers share the same length. At the same time, they should have more digits as the same like 12949 and 12961 to avoid models that can solve the problem by only counting the length or comparing the first digit. For scientific notation, we ensure 70% pairs of compared numbers with the same exponential part so that models cannot directly get the answer without comparing the significand part. For fractions, we ensure the numbers are both less than one, avoiding the model can just compare them with 1 to get more than 50% accuracy.
- For “to_float-Fraction”, we require that the fraction can be converted into a finite decimal, that is the denominator contains only factors 2 and 5.
- For “add/sub-Scientific”, we require the exponential part of each number to have a difference less than 5 to make sure that the generated answer will not be too long.

The pre-processing could introduce additional duplicated data, so we implement a post-filtering step to remove duplicates and ensure data integrity.

A.1.8 METRICS

For digit match, we should first align the numbers. For the integers and integer parts in floats, the numerator and denominator of fractions, and the exponential part of the scientific notation, we use the right alignment. For the decimal part in floats (as well as the in the significand part in scientific notation), we use the left alignment.

For dlength, we first measure the difference of each part of a number and then add the absolute values up.

Besides the average metrics in each range, we also present the following metrics: *well-learned digits* and *performance-preserving digits* to demonstrate the model’s upper and lower performance limits on length. These represent the maximum number of digits that can maintain over 90% and 10% accuracy, respectively. (For digit match, the thresholds are set to 90% and 50%, and for dlength, where smaller is better, the thresholds are 0.1 and 1).

We ensure that there is no duplicated sample in dataset, so for some range, the test samples could be less than 1000. We also omit 1 digit or some 2 digit test in our testbed to make sure that unit rules can be included in a training set.

A.2 PROMPTS AND OTHER DETAILS TO TEST BASELINE MODELS

For all models in our test, we first provide a “format prompt” describing the expected return format (and avoiding models generating complex CoT), and a “task prompt” describing the task. We use some easy problems to ensure powerful models (gpt-4o-mini and Llama-3.1-8B) can correctly understand the tasks and expected return format by the prompts. The expected return representation of each task is referred to in Appendix A.1.4.

The **format prompt** based on the expected return type of the task is as follows:

- **Integer:** Directly return the answer as an integer without any comma separator, like 123 .
- **float:** Directly return the answer as a float without any comma separator, like 10.4 .
- **Fraction:** Directly return the answer as an ****irreducible**** fraction without any comma separator, like 7/13 .
- **Scientific Notation:** Directly return the answer as a scientific notation without any comma separator, like 1.23e4 . The float part should be in the range [1, 10).

The **task prompts** are listed as follows where $\langle a \rangle$ and $\langle b \rangle$ are numbers.

- **Add:** Add two numbers: $\langle a \rangle + \langle b \rangle =$
- **Sub:** Subtract two numbers: $\langle a \rangle - \langle b \rangle =$
- **Multiply:** Multiply two numbers: $\langle a \rangle * \langle b \rangle =$
- **Truediv:** Divide two numbers and return the result as a fraction. $\langle a \rangle / \langle b \rangle =$
- **Floordiv:** Divide two numbers and return the result as an integer. $\langle a \rangle // \langle b \rangle =$
- **Mod:** Divide two numbers and return the remainder. $\langle a \rangle \% \langle b \rangle$
- **Max:** Get the maximal number: $\langle a \rangle$ and $\langle b \rangle =$
- **Min:** Get the minimal number: $\langle a \rangle$ and $\langle b \rangle =$
- **Digit max:** Compare two numbers digit by digit and return the larger digit at each position, treating any missing digits as 0. $\langle a \rangle$ and $\langle b \rangle =$
- **Digit min:** Compare two numbers digit by digit and return the smaller digit at each position, treating any missing digits as 0. $\langle a \rangle$ and $\langle b \rangle =$
- **Digit add:** The task is to add two given numbers digit by digit and return the result modulo 10 (ignoring carry), treating any missing digits as 0. $\langle a \rangle$ digit add $\langle b \rangle =$
- **Get digit:** Get the digit at the given position (from left to right, starting from 0). $\langle a \rangle$ at position $\langle b \rangle =$
- **Length:** The total number of digits of $\langle a \rangle =$

- **Count:** Count the number of the given digit in the given number: `<a>` count the occurrence time of digit `<b>` =
- **To_float:** Convert the number to float: `<a>` =
- **To_scient:** Convert the number to scientific notation: `<a>` =
- **Sig_fig:** Convert the number to scientific notation: `<a>` and keep significant figures as `<b>`.

Notice that all prompts are ended with an “=” so that we can easily separate the input question and the generation of models. When we use the texts in supervised finetuning (SFT), the context before the “=” is not involved in the loss calculation.

For GPT-4o and GPT-4o-mini, we also add a system message as follows and use the aforementioned question as user message:

You are a capable math assistant. Return your solution without any process in the format: The answer is [YOUR ANSWER]. The final answer must strictly match the format `<regex>`.

where the `<regex>` is a regular expression based on the expected return format:

- **Integer:** `r"\d+"`
- **Float:** `r"\d+\.\d+"`
- **Fraction:** `r"\d+/\d+"`
- **Scientific Notation:** `r"\d+\.\d+e\d+"`

We use the models expect GPT from huggingface and use the default tokenizer, model and generation configuration provided by the models. We test GPT-4o and GPT-4o-mini by the OpenAI API, where GPT-4o means gpt-4o-2024-0806 and GPT-4o-mini means GPT-4o-mini-2024-07-18. For Qwen2-72B and Llama-3.1-70B, we additionally use 4-bit quantization but we also test several samples without quantization and ensure this quantization does not affect generation quality.

We retrieve the first match of the corresponding regular expression after the “=” as the answer. If there is no retrieve, we use an empty answer to calculate the metrics, where exact match and digit match is both zero and the dlength is the total length of the groundtruth number.

A.3 FULL TEST RESULTS OF LLMs

We show the full NUPA Test results in Figures 5 (exact match), 6 (digit match), 7 (dlength) and Table 7, 8, 9 (well-learned digits and performance-preserving digits for each metrics).

With the detailed metrics, we can more clearly understand the behavior of some models on some tasks. For example, we find that the “exact match” and “digit match” of some models like Qwen-2 and GPT-4o on the “integer-max” task are similar, suggesting that when the models know which one is correct, they can always copy the answer from question correctly. So the wrong answer comes from incorrect comparison. Another example is the Llama-2 performance on max-hard. Because the length of two input numbers and the groundtruth answer in the max-hard task are all the same, most models show less dlength on this task suggesting they know that “the answer should have the same length of inputs”, but we find Llama-2 shows dlength approximately equal to the average length in the range, suggesting that Llama-2 cannot generate a valid answer on this task. These are just a few examples to illustrate how more detailed metrics can help us gain a deeper understanding of model behavior. There are many possible conclusions, but there are too many to list here.

A.3.1 FEW-SHOT LEARNING

To ensure the output format of models is as precise as possible, we employ 5-shot learning. For each task, we select one sample from 5 different lengths respectively and test the few-shot performance. Table 10 summarizes the exact match score performance across three selected tasks and Figure 8 shows more tasks. Notably, providing an explicit output format results in general performance improvements across tasks and input lengths. In most tasks, the models can usually produce accurately formatted